SENG 691

AI AGENT COMPUTING

HW3 – AGENT DETERMINISTIC PLANNING

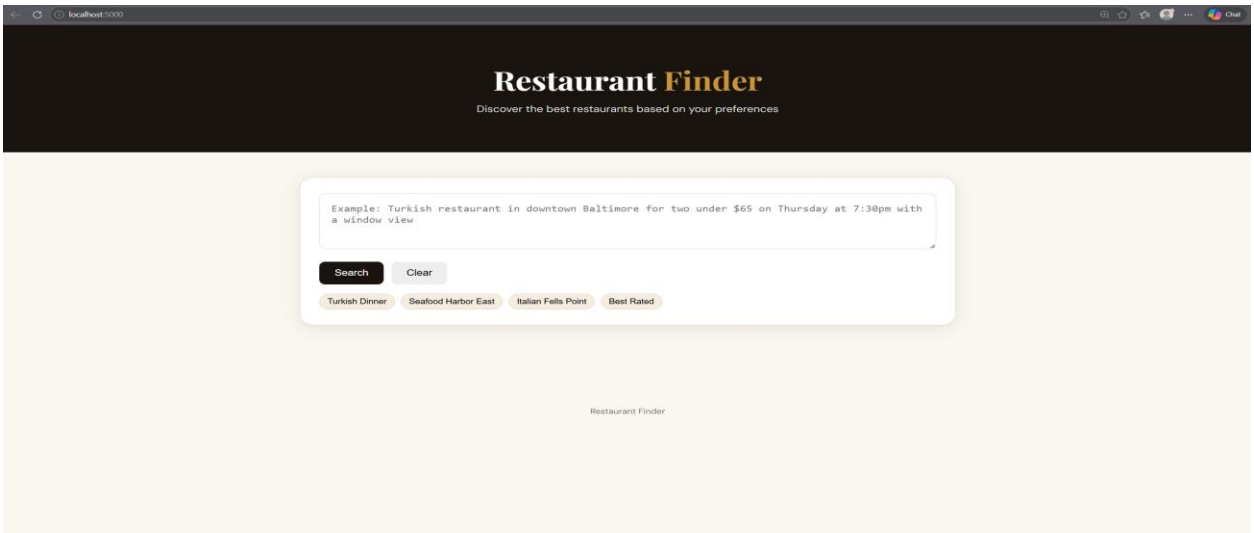
Submitted by

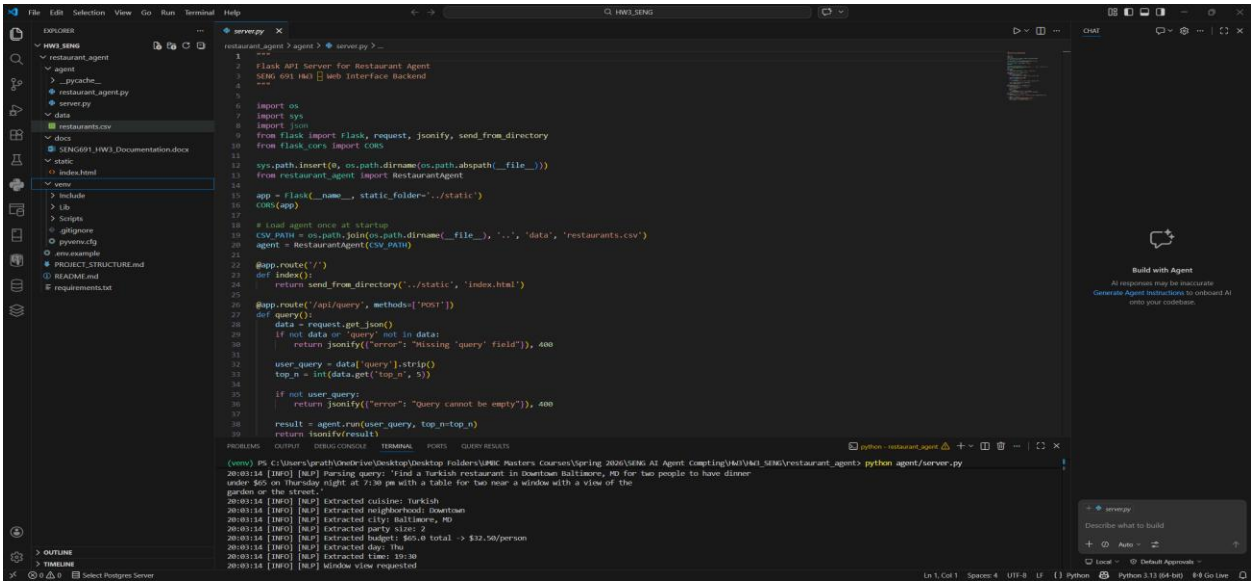
Prathyusha Harish Kumar
JB24771

Objective: *Build an AI agent for restaurant recommendations. Create a simple intelligent agent that finds restaurants based on user preferences and constraints. Demonstrate concepts and techniques from Searching for Solutions, Reasoning with Constraints, and Deterministic Planning.*

This project implements an intelligent restaurant recommendation agent using goal-based and utility-based reasoning architecture. The agent accepts natural language queries from a user and determines the best restaurant matches by applying a deterministic planning pipeline consisting of natural language parsing, constraint filtering, and utility-based ranking.

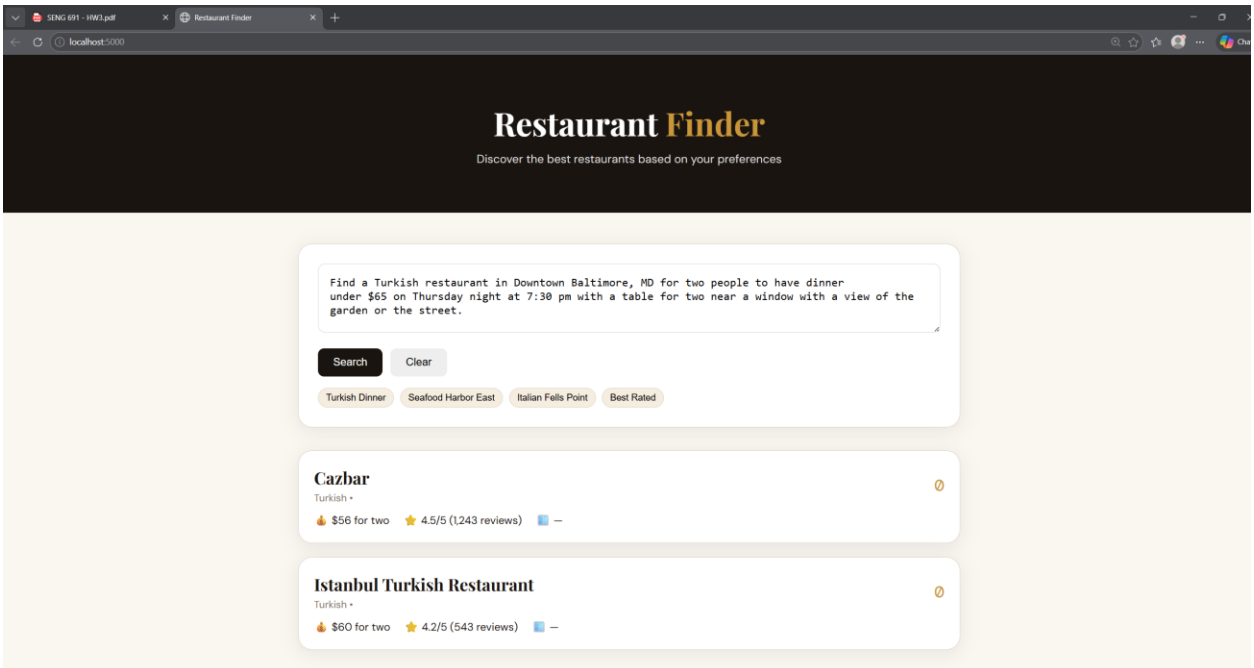
The system is designed to assist users in finding restaurants that satisfy specific constraints such as cuisine type, location, budget, dining time, and seating preferences. The agent is implemented using Python and deployed as a web-based interface using Flask, where it allows us to submit queries and receive ranked restaurant recommendations in real time.





Scenario: *"Find a Turkish restaurant in Downtown Baltimore, MD for two people to have dinner under $65 on Thursday night at 7:30 pm with a table for two near a window with a view of the garden or the street."*

I have developed a system designed to solve the above query scenario provided in the homework assignment document. The agent will interpret this natural language query, extract relevant constraints, and determine which restaurants satisfy those conditions.



Tasks:

1. Define the states, actions, and goals.

2. Use a pipeline approach with but not limited to the methods below:

- *Natural Language Processing (NLP) for understanding the query.*
- *Search API or dataset (e.g., Yelp API or open data) for retrieving restaurants.*
- *Filtering and ranking logic (location = Downtown Baltimore, MD; cuisine = Turkish; price ≤ $65).*

3. Simulate agent reasoning: show logs of events and decisions made.

4. Visualize or output final ranked results.

5. The agent should explain how the ranked results match the criteria i.e. location: downtown Baltimore, cuisine: Turkish, price: dinner for two under $65, view: window view.

STATES

To track how the agent progresses while solving the problem, I created an **AgentState dataclass**. This class stores the internal status of the agent as it moves through different stages of the reasoning process.

The agent operates through three main states:

- **Initial State -** In the initial state, the system has just received the user query. At this stage the database is loaded, but no parsing, filtering, or ranking has been performed yet.
- **Intermediate States -** After receiving the query, the agent moves through several intermediate steps. First, the natural language query is parsed to extract structured

information such as cuisine, location, price, and time. Then the system searches the restaurant database and applies constraint filtering. Finally, the remaining restaurants are ranked using the utility scoring function.

- **Goal State -** The goal state is reached when the agent successfully generates the final recommendations. At this point the system has identified the best matching restaurants and returns the **top-N ranked results along with explanations** for why each restaurant satisfies the query.

ACTIONS

To move between these states, the agent performs a series of actions. These actions represent the reasoning steps the agent takes to process the query and produce recommendations.
The main actions implemented in the system are:

- **PARSE_QUERY -** I implemented this action to convert the raw natural language query into a structured **AgentGoal object**. The parser extracts information such as cuisine type, location, budget, time, and seating preferences.
- **LOAD_DATA -** In this step, the agent loads the restaurant dataset from the CSV file. This dataset contains information about restaurants including price, rating, location, and seating options.
- **FILTER(constraint) -** The filtering action removes restaurants that do not satisfy the user's constraints. Each constraint is applied sequentially, such as cuisine match, location match, price limit, and opening hours.
- **RELAX_CONSTRAINT -** If no restaurants satisfy all constraints, the system relaxes one of the constraints (usually location) so that the agent can still provide useful recommendations instead of returning no results.
- **SCORE(restaurant, goal) -** For each restaurant that passes the filtering stage, the agent calculates a **utility score** based on several factors such as rating, review count, price efficiency, and view preferences.
- **SORT(candidates) -** After scoring all restaurants, the candidates are sorted in descending order based on their utility score so the best options appear first.
- **EXPLAIN(restaurant, goal) -** Finally, the agent generates a natural language explanation describing why each restaurant matches the user's requirements.

GOALS

The main goal of the agent is to identify the best restaurant options that satisfy the user's request.
To achieve this, the agent performs the following objectives:

- Ensure that all **hard constraints** from the user query are satisfied, including cuisine type, location, price limit, day, and time.
- Use a **utility scoring function** to evaluate the remaining restaurants based on soft preferences such as views, ratings, number of reviews, price efficiency, and reservation availability.
- Return the **top-N ranked restaurant recommendations** along with clear explanations showing how each restaurant matches the user's criteria.

By combining constraint filtering and utility-based ranking, the agent can provide accurate and transparent recommendations that closely match the user's preferences.

Agent Type

The agent is a hybrid Goal-Based + Utility-Based agent:

- **Goal-Based reasoning:** The agent parses the query to define an explicit goal state (desired cuisine, location, price, time, seating). Filtering acts as constraint satisfaction, a restaurant must satisfy all hard constraints.
- **Utility-Based reasoning:** When multiple restaurants satisfy all hard constraints, the agent assigns a utility score (0–100) based on weighted soft preferences including rating, review volume, price efficiency, view match, and reservation availability.

PLANNING PIPELINE (DETERMINISTIC)

The agent follows a 5-phase deterministic planning pipeline:

Phase 1: PERCEIVE	NLP parser extracts structured constraints from raw text query using regex-based entity recognition for cuisine, location, price, time, day, party size, and seating preferences.
Phase 2: GOAL SET	Parsed constraints are encoded into an AgentGoal dataclass. The AgentState tracks pipeline progress (query_received, query_parsed, filtered, ranked, results_ready).
Phase 3: SEARCH	All 50 restaurants are loaded from the CSV database into memory. This acts as the initial state space for the planning problem.
Phase 4: FILTER (CSP)	Hard constraints are applied sequentially: cuisine match, location match, price ≤ budget, open on requested day, and open at requested time. Each filter reduces the candidate set.
Phase 5: RANK & EXPLAIN	Remaining candidates are scored using the utility function. Top-N results are returned with auto-generated natural language explanations per criterion.

AGENT REASONING

To demonstrate how the AI agent makes decisions, I simulated the reasoning process by running the system with the example query provided in the assignment.
During execution, the agent generates a reasoning log that records each step of the decision-making pipeline. These logs allow us to observe how the agent interprets the query, filters the restaurant database, ranks the results, and selects the best recommendation.
The reasoning process follows a deterministic pipeline where the agent performs several sequential actions.

1. NLPQueryParser

I implemented the NLPQueryParser module to extract structured information from the user's natural language query. This component uses rule-based Natural Language Processing with Python regular expressions and keyword dictionaries to detect important entities in the input sentence.

- Cuisine type (Turkish, Italian, Japanese, Greek, etc.)
- Location / neighborhood (Downtown Baltimore, Fells Point, Harbor East, etc.)
- Budget in dollars (patterns: '$65', 'under $65', '65 dollars')
- Party size (patterns: 'for two', 'party of 3', 'table for 2')
- Day of week and time (Thursday, 7:30 pm)
- Seating and view preferences (window, garden, street, harbor, rooftop)

After extracting these parameters, the parser converts them into a structured **AgentGoal object**, which represents the goal state that the agent attempts to satisfy. I designed this module using simple and deterministic using regular expressions and advanced NLP tools such as **spaCy Named Entity Recognition** to improve accuracy for complex or ambiguous queries.

```
Pipeline:
  NLP Parser -> Filter Engine -> Ranking Engine -> Explanation Engine
"""

def __init__(self, csv_path: str):
    self.df = pd.read_csv(csv_path)
    self.parser = NLPQueryParser()
    self.explainer = ExplanationEngine()
    log.info(f"[AGENT] Loaded {len(self.df)} restaurants from {csv_path}")

def run(self, query: str, top_n: int = 5) -> dict:
    print("\n" + "="*70)
    print("RESTAURANT RECOMMENDATION AGENT - SENG 691 HW3")
    print("="*70)
    print(f"Query: {query}")
    print("="*70 + "\n")

    state = AgentState(step="NLP")

    state.record("Starting NLP query parsing...")
    goal = self.parser.parse(query)
    state.record(
        f"Parsed goal: cuisine={goal.cuisine}, location={goal.location}, "
        f"party={goal.party_size}, budget=${goal.max_total}, "
        f"day={goal.day}, time={goal.time}, "
        f"window={goal.window_view}, views={goal.view_types}"
    )
```

2. RestaurantDatabase

I created the **RestaurantDatabase** module to manage the restaurant dataset used by the agent. This module loads the **50-restaurant CSV database** and converts each row into a structured Python dictionary that the agent can process efficiently.

During the loading process, I implemented several preprocessing steps:

- Numeric fields such as **price, rating, and review counts** are converted into numeric types.
- List fields such as **seating options and available views** are parsed from semicolon-delimited strings.
- Boolean attributes such as **reservation availability and opening days** are normalized to True/False values.

This database contains restaurants from approximately **15 neighborhoods across Baltimore and surrounding areas**, providing sufficient variety for testing different user queries.

By loading the CSV file into memory, the agent can quickly search, filter, and rank restaurants

without needing an external API.

```
restaurants.csv
restaurant_agent > data > restaurants.csv
id,name,cuisine,location,neighborhood,city,state,address,price_per_person,rating,reviews,open_days,dinner_start,dinner_end,window_view,view_type,table_for_two,
1,Cazbar,Turkish,Downtown,Downtown,Baltimore,MD,316 N Charles St Baltimore MD 21201,28,4.5,1243,Mon-Sun,17:00,23:00,True,street,True,False,True,410-528-1222,ca
2,Ambassador Dining Room,Indian,Roland Park,Roland Park,Baltimore,MD,3811 Canterbury Rd Baltimore MD 21218,35,4.3,876,Tue-Sun,17:30,22:00,True,garden,True,True
3,Wit & Wisdom,American,Inner Harbor,Inner Harbor,Baltimore,MD,200 International Dr Baltimore MD 21202,45,4.4,2341,Mon-Sun,17:00,22:30,True,harbor,True,True,Tr
4,Magdalena,Mediterranean,Mount Vernon,Mount Vernon,Baltimore,MD,617 W Lexington St Baltimore MD 21201,38,4.6,987,Tue-Sat,17:30,22:00,True,garden,True,False,Tr
5,Woodberry Kitchen,American,Clipper Mill,Clipper Mill,Baltimore,MD,2010 Clipper Park Rd Baltimore MD 21211,42,4.5,3210,Tue-Sun,17:00,22:00,True,garden,True,Tr
6,The Prime Rib,American,Mount Vernon,Mount Vernon,Baltimore,MD,1101 N Calvert St Baltimore MD 21202,55,4.7,4521,Mon-Sat,17:00,23:00,False,none,True,False,True
7,Charleston,Southern,Harbor East,Harbor East,Baltimore,MD,1000 Lancaster St Baltimore MD 21202,60,4.8,2876,Tue-Sat,17:30,22:00,True,street,True,False,True,410
8,Tagliata,Italian,Harbor East,Harbor East,Baltimore,MD,1012 Fleet St Baltimore MD 21202,40,4.4,1654,Mon-Sun,17:00,22:30,True,street,True,False,True,410-558-08
9,Petit Louis Bistro,French,Roland Park,Roland Park,Baltimore,MD,4800 Roland Ave Baltimore MD 21210,36,4.5,1432,Tue-Sun,17:30,22:00,True,street,True,True,True,
10,Ekiben,Asian Fusion,Federal Hill,Federal Hill,Baltimore,MD,1622 Fleet St Baltimore MD 21231,18,4.6,2341,Mon-Sun,11:00,22:00,False,none,True,False,False,410-
11,Birroteca,Italian,Hampden,Hampden,Baltimore,MD,1520 Clipper Rd Baltimore MD 21211,28,4.3,1123,Tue-Sun,17:00,22:00,True,garden,True,True,False,443-708-1935,b
12,Ouzo Bay,Greek Seafood,Harbor East,Harbor East,Baltimore,MD,1000 Lancaster St Baltimore MD 21202,48,4.5,1876,Mon-Sun,17:00,23:00,True,street,True,True,True,
13,Pabu,Japanese,Inner Harbor,Inner Harbor,Baltimore,MD,725 Aliceanna St Baltimore MD 21202,55,4.4,987,Tue-Sat,17:30,22:30,True,street,True,False,True,443-835-
14,Cinghiale,Italian,Harbor East,Harbor East,Baltimore,MD,822 Lancaster St Baltimore MD 21202,42,4.6,2109,Tue-Sat,17:30,22:00,True,street,True,False,True,410-5
15,Alma Cocina Latina,Latin American,Station North,Station North,Baltimore,MD,415 N Charles St Baltimore MD 21201,30,4.7,1543,Tue-Sat,17:00,22:00,True,street,T
16,Artifact Coffee,Cafe,Clipper Mill,Clipper Mill,Baltimore,MD,1500 Union Ave Baltimore MD 21211,15,4.4,876,Mon-Sat,07:00,18:00,True,garden,True,False,False,41
17,Encantada,Mexican,Hampden,Hampden,Baltimore,MD,1622 W 41st St Baltimore MD 21211,25,4.2,765,Tue-Sun,17:00,22:00,False,none,True,True,False,410-366-1015,enca
18,The Food Market,American,Hampden,Hampden,Baltimore,MD,1017 W 36th St Baltimore MD 21211,32,4.4,2345,Mon-Sun,17:00,22:30,True,street,True,False,False,410-366
19,Kyodai,Japanese,Mount Vernon,Mount Vernon,Baltimore,MD,211 E Fayette St Baltimore MD 21202,28,4.3,654,Mon-Sun,11:30,22:00,False,none,True,False,False,410-75
20,Bottega,Italian,Hampden,Hampden,Baltimore,MD,905 W 36th St Baltimore MD 21211,35,4.5,1098,Tue-Sun,17:00,22:00,True,street,True,False,True,410-235-7666,botte
21,Istanbul Turkish Restaurant,Turkish,Downtown,Downtown,Baltimore,MD,502 W Pratt St Baltimore MD 21201,30,4.2,543,Mon-Sun,11:30,22:30,True,street,True,False,F
22,Bosphorus Turkish Cuisine,Turkish,Federal Hill,Federal Hill,Baltimore,MD,604 S Federal Hill Blvd Baltimore MD 21230,27,4.0,432,Tue-Sun,16:00,22:00,True,gard
23,Topkapi Grill,Turkish,Mount Vernon,Mount Vernon,Baltimore,MD,900 N Charles St Baltimore MD 21201,32,4.3,678,Mon-Sat,17:00,22:30,True,street,True,False,True,
24,Anatolia Turkish Kitchen,Turkish,Inner Harbor,Inner Harbor,Baltimore,MD,201 E Pratt St Baltimore MD 21202,29,4.1,398,Mon-Sun,12:00,23:00,True,harbor,True,Fa
25,The Capital Grille,Steakhouse,Inner Harbor,Inner Harbor,Baltimore,MD,500 Light St Baltimore MD 21202,65,4.6,3210,Mon-Sun,17:00,22:30,False,none,True,False,T
26,Brewer's Art,American,Mount Vernon,Mount Vernon,Baltimore,MD,1106 N Charles St Baltimore MD 21201,25,4.3,2109,Mon-Sun,16:00,02:00,True,street,True,False,Fal
27,Clavel,Mexican,Remington,Remington,Baltimore,MD,225 W 23rd St Baltimore MD 21211,22,4.6,1876,Tue-Sun,17:00,22:00,False,none,True,True,False,443-900-8983,cla
28,Fork and Wrench,American,Canton,Canton,Baltimore,MD,2322 Boston St Baltimore MD 21224,35,4.4,1654,Tue-Sun,17:00,22:00,True,street,True,False,True,443-759-93
29,Pappas Restaurant,Seafood,Parkville,Parkville,Baltimore,MD,1725 Taylor Ave Baltimore MD 21234,38,4.3,987,Mon-Sun,11:00,22:00,True,garden,True,True,False,410
30,LP Steamers,Seafood,South Baltimore,Locust Point,Baltimore,MD,1100 E Fort Ave Baltimore MD 21230,30,4.5,2341,Mon-Sun,11:00,22:00,False,none,True,True,False,
31,Heavy Seas Alehouse,American,Downtown,Downtown,Baltimore,MD,1300 Bank St Baltimore MD 21231,26,4.2,1234,Mon-Sun,11:00,23:00,True,street,True,True,False,410-
32,Smoke BBQ,BBQ,Downtown,Downtown,Baltimore,MD,801 S Bond St Baltimore MD 21231,24,4.1,876,Mon-Sun,11:00,22:00,False,none,True,True,False,410-528-1000,smokeba
33,Taqueria El Sabor,Mexican,Fells Point,Fells Point,Baltimore,MD,1701 Aliceanna St Baltimore MD 21231,18,4.0,543,Mon-Sun,10:00,22:00,False,none,True,False,Fal
34,Fleet Street Kitchen,American,Harbor East,Harbor East,Baltimore,MD,1012 Fleet St Baltimore MD 21202,44,4.5,1432,Mon-Sat,17:00,22:00,True,street,True,False,T
35,Salt Tavern,American,Upper Fells Point,Upper Fells Point,Baltimore,MD,2127 E Pratt St Baltimore MD 21231,32,4.4,987,Tue-Sun,17:00,22:00,True,street,True,Fal
36,Azumi,Japanese,Harbor East,Harbor East,Baltimore,MD,725 Aliceanna St Baltimore MD 21202,52,4.6,1765,Mon-Sun,17:30,23:00,True,street,True,False,True,443-220-
37,Kooper's Tavern,American,Fells Point,Fells Point,Baltimore,MD,1702 Thames St Baltimore MD 21231,26,4.2,2109,Mon-Sun,11:00,02:00,True,street,True,True,False,
38,Thames Street Oyster House,Seafood,Fells Point,Fells Point,Baltimore,MD,1728 Thames St Baltimore MD 21231,40,4.5,1654,Tue-Sun,17:00,22:00,True,street,True,T

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   QUERY RESULTS
python - restaurant_agent

(venv) PS C:\Users\prath\OneDrive\Desktop\Desktop Folders\UMBC Masters Courses\Spring 2026\SENG AI Agent Compting\HW3\HW3_SENG\restaurant_agent> python agent/server.py
```

3. FilterEngine (Constraint Satisfaction)

I implemented the FilterEngine to enforce the hard constraints specified in the user query. This component performs Constraint Satisfaction Problem (CSP) filtering by removing restaurants that do not meet the required conditions.

The filtering process is performed sequentially using list comprehensions, allowing each constraint to reduce the candidate restaurant list.

Filters applied: Cuisine match, Location match, Price ≤ max_for_two, Open on day, Open at time

```python
    state.step = "FILTERING"
    engine = RestaurantFilterEngine(self.df, state)
    filtered = engine.apply_filters(goal)

    if filtered.empty:
        print("\nNo restaurants matched all criteria. Relaxing view filter...\n")
        state.record("No results found - relaxing window view filter")
        goal.window_view = False
        goal.view_types = []
        filtered = engine.apply_filters(goal)

    state.step = "RANKING"
    ranked = engine.rank(filtered, goal)
    top = ranked.head(top_n)

    print("\n" + "-"*70)
    print("PIPELINE SUMMARY")
    print("-"*70)
    print(f"  Total restaurants in DB  : {state.total_restaurants}")
    print(f"  After cuisine filter     : {state.after_cuisine_filter}")
    print(f"  After location filter    : {state.after_location_filter}")
    print(f"  After price filter       : {state.after_price_filter}")
    print(f"  After hours filter       : {state.after_hours_filter}")
    print(f"  After view filter        : {state.after_view_filter}")
    print(f"  Final candidates ranked  : {state.final_count}")
    print(f"  Showing top              : {len(top)}")
    print("-"*70)

    print(f"\nTOP {len(top)} RECOMMENDATIONS\n")
    results = []
    for rank, (_, row) in enumerate(top.iterrows(), 1):
        explanation = self.explainer.explain(row, goal)
        print(f"#{rank}  {row['name']}")
        print(f"    Score: {row['total_score']:.1f}/100")
        print(f"    {explanation}")
        print()
        results.append({
            "rank": rank,
            "name": row["name"],
            "cuisine": row["cuisine"],
            "address": row["address"],
```

4. RankingEngine (Utility Function)

```python
    state.step = "FILTERING"
    engine = RestaurantFilterEngine(self.df, state)
    filtered = engine.apply_filters(goal)

    if filtered.empty:
        print("\nNo restaurants matched all criteria. Relaxing view filter...\n")
        state.record("No results found - relaxing window view filter")
        goal.window_view = False
        goal.view_types = []
        filtered = engine.apply_filters(goal)

    state.step = "RANKING"
    ranked = engine.rank(filtered, goal)
    top = ranked.head(top_n)

    print("\n" + "-"*70)
    print("PIPELINE SUMMARY")
    print("-"*70)
    print(f"  Total restaurants in DB  : {state.total_restaurants}")
    print(f"  After cuisine filter     : {state.after_cuisine_filter}")
    print(f"  After location filter    : {state.after_location_filter}")
    print(f"  After price filter       : {state.after_price_filter}")
    print(f"  After hours filter       : {state.after_hours_filter}")
    print(f"  After view filter        : {state.after_view_filter}")
    print(f"  Final candidates ranked  : {state.final_count}")
    print(f"  Showing top              : {len(top)}")
    print("-"*70)

    print(f"\nTOP {len(top)} RECOMMENDATIONS\n")
    results = []
    for rank, (_, row) in enumerate(top.iterrows(), 1):
        explanation = self.explainer.explain(row, goal)
        print(f"#{rank}  {row['name']}")
        print(f"    Score: {row['total_score']:.1f}/100")
        print(f"    {explanation}")
        print()
        results.append({
            "rank": rank,
            "name": row["name"],
            "cuisine": row["cuisine"],
            "address": row["address"],
```

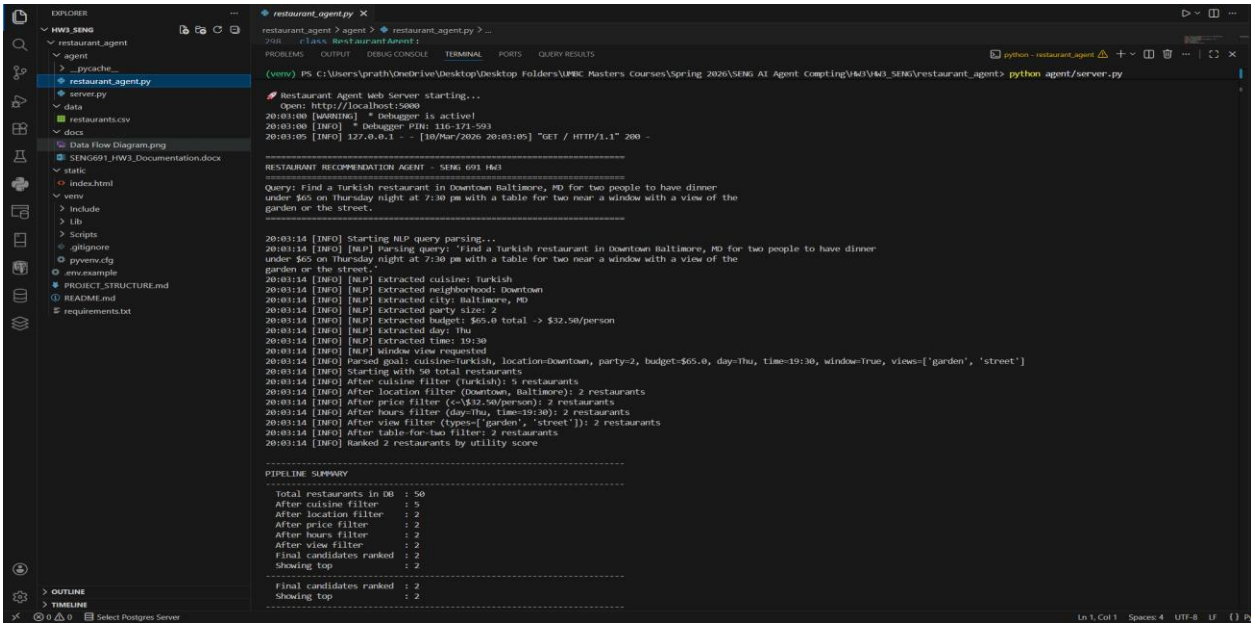
Computes a utility score out of 100 points:

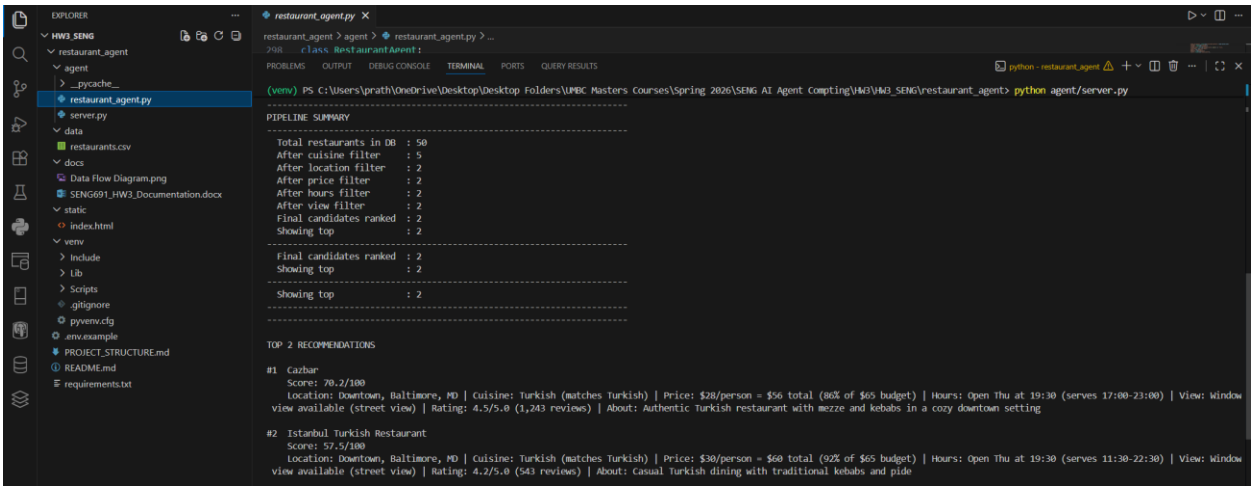
Rating (max 40 pts)	(rating / 5.0) × 40 - rewards highly rated restaurants
Review Volume (max 15 pts)	log10(reviews) / log10(5000) × 15 - rewards credible review counts
Price Efficiency (max 20 pts)	(1 - price/budget) × 20 + 10 - rewards staying well under budget
View Match (max 15 pts)	8 pts for window/street view, 7 pts per preferred view matched
Reservation (max 10 pts)	Bonus if restaurant accepts reservations

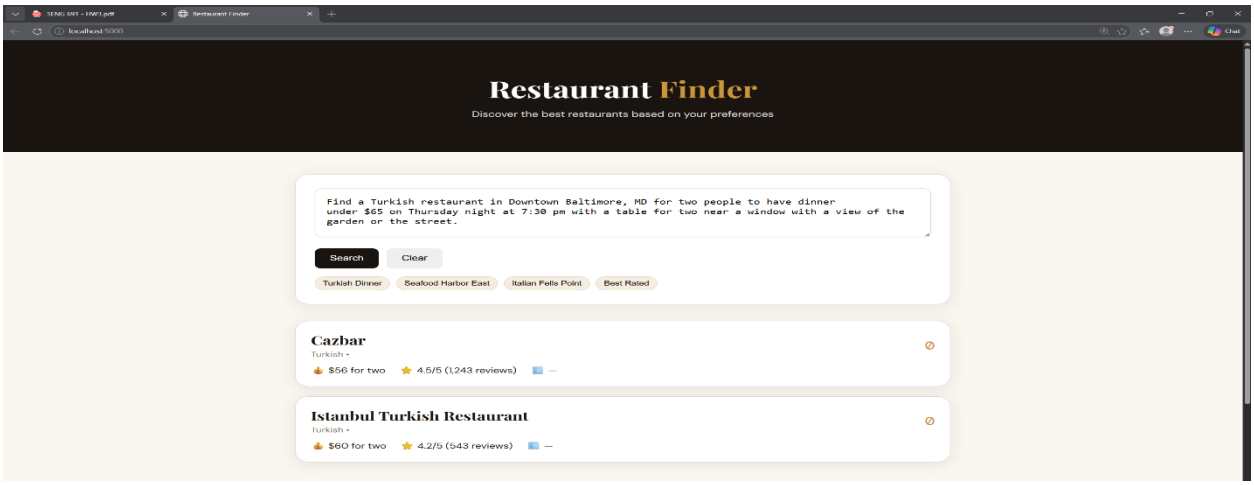
5. Explainer

I implemented the **Explainer module** to generate natural language explanations describing why each restaurant was recommended. This component maps the results of the filtering and ranking stages back to the original user query.

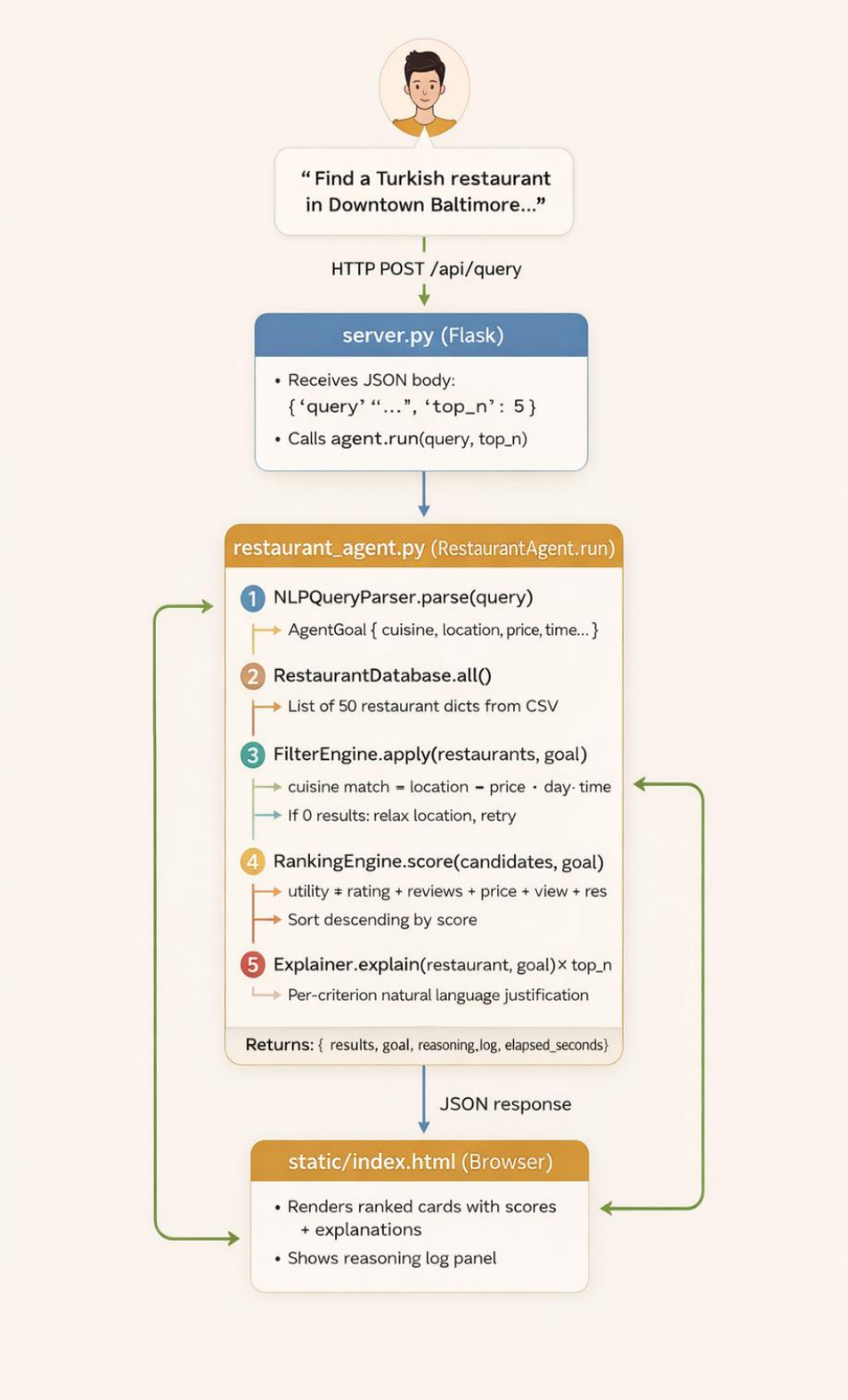
TERMINAL AND WEBPAGE SCREENSHOTS [Event Logs, Explaination and Decision]







DATA FLOW DIAGRAM [INSTRUCTIONS]



TOOLS USED

Core Agent	Python 3.9+
NLP Module	Regex + keyword dictionaries (extensible to spaCy/OpenAI)
Database	CSV (50 restaurants) + Python csv.DictReader
Filtering	Custom FilterEngine with sequential CSP constraint application
Ranking	Custom utility function with 5 weighted dimensions
Web Server	Flask 3.0 + flask-cors
Web UI	Vanilla HTML5 / CSS3 / JavaScript (no framework dependencies)
Deployment	Local server (localhost:5000)

BUILDING/DEPLOYMEN INSTRUCTIONS

✓ **Prerequisites**

It is required for the following software to be installed installed:

- Python 3.9 or later
- pip (Python package manager)
- Terminal or Command Prompt
- A web browser (Chrome, Edge.)

✓ **Installation**

1. Download or clone the project folder to your computer.
2. Navigate to the project directory and install the required dependencies.

```
cd restaurant_agent
pip install flask flask-cors pandas
```

These libraries are required for:

- Running the Flask web server
- Handling API requests
- Processing the restaurant dataset

No external API keys are required because the agent uses the **bundled 50-restaurant CSV database**.

✓ **Run CLI (Command Line Interface)**

The agent can be executed directly from the terminal.
Run the following command:

```
python agent/restaurant_agent.py
```

This runs the default HW3 scenario query and prints the following information in the terminal:

- Parsed query parameters
- Filtering pipeline logs
- Utility ranking results
- Final restaurant recommendations

To run a custom query, use:

python agent/restaurant_agent.py "Find a seafood restaurant in Harbor East with waterfront views for two under $120"

The agent will process the query and display the ranked results in the terminal.

✓ **Run Web User Interface**

The project also provides a web-based interface for interacting with the agent.
Start the Flask server using:

python agent/server.py

After starting the server, open a web browser and go to:
http://localhost:5000

CONCLUSION

In this homework, I developed a Restaurant AI Recommendation Agent that can understand natural language queries and recommend restaurants based on user preferences. The agent combines goal-based reasoning and utility-based ranking to identify the best restaurant options that satisfy the user's constraints. The system processes a query through several stages including NLP parsing, database search, constraint filtering, utility scoring, and explanation generation. By applying these steps, the agent is able to reduce a large restaurant dataset into a small set of highly relevant recommendations.

To make the system easier to use, I also implemented a web interface using Flask, HTML, CSS, and JavaScript, which allows users to enter queries and view ranked restaurant results along with reasoning logs. The reasoning logs provide transparency by showing how the agent interprets the query and makes decisions during each step of the pipeline.

Overall, this project demonstrates how an intelligent agent can apply AI concepts such as PEAS modeling, constraint satisfaction, and utility-based decision making to solve a real-world recommendation problem. The system is modular and can be extended in the future by integrating more advanced NLP models, larger restaurant datasets

Tools to consider but not limited to (you may use any of these or other tools and APIs to design and build your agent):

- *LangChain/CrewAI (for agent framework)*
- *OpenAI API (or spaCy for NLP)*
- *Pandas for filtering*
- *Yelp API or custom CSV dataset*

Expected outcome and deliverables:

- *Script or custom-built agent binary and instructions for building it*
- *Source code and documentation including written description of agent architecture (goal-based, utility-based reasoning, and/or other techniques).*
- *Video recording of a demonstration of the agent being compiled, running, and producing the results.*

Rubric:

- Full Credit (100%): Student builds a fully working AI agent that correctly interprets the query, filters data, ranks results, and clearly explains reasoning in both code and demo.
- Partial Credit (50%): Student builds a partially working agent with limited reasoning or missing parts of the pipeline.
- Minimal Credit (25%): Student submits an incomplete or non-functional agent with little explanation or missing demonstration.
- No Credit (0%): Student does not submit a working agent or fails to provide any meaningful code, documentation, or demonstration.